Offline PO Agent 10/16 State

High Level

It's a Next.js app on Vercel that reads purchase orders arriving by email (PDF, spreadsheet or plain text) and turns each one into a BigCommerce B2B Edition **order** (pay on account) or **quote** (with a checkout link). It replies on the same email thread. It applies the same controls the storefront would: who the buyer is, their contract pricing, their catalog, their credit, and their role.

The project can be accessed here:

Vercel: https://offline-po-app.vercel.app/

BigCommerce: https://store-ybhg7gjokp.mybigcommerce.com/manage/app/66672

Github: https://github.com/wayne-stratbucker-silk/offline-po-app

Current Implementation

Most of the implemented features can be found on “Simulator” page. There, a simulation of a user flow is implemented, and goes as follows:

1. A scenario is chosen (e.g., quote request)
2. The model (Claude Haiku 5.5 via Anthropic API Key) will parse the information and output a JSON of relevant information such as product name, SKU, quantity, and price
3. After this step, everything is done programmatically (needs to change)
    a. The relevant information is confirmed through BigCommerce via **REST** calls
        i. Resolve SKUs → price → validate → decide → act → reply
    b. When confirmed, the project uses lib/po-agent/rules to determine the logic, and these rules are based on human-validated use cases in petrovalve-offline-po-decision-rules.md

    c. In a base case, once the product information is validated, a response email and decision trail are outputted to the simulator page for the user to see
    d. After this step, the user can optionally respond, starting a reply on the same thread
4. Redis stores each email thread’s whole session

Acting on decisions

Clarify. The agent asks one combined question, at most 2 rounds, then escalates. Two cases come with ready-made answers:

- **Parent SKU:** the reply lists each child variant with its contract price.
- **Price mismatch:** the reply offers CONFIRM (order at the contract price) or QUOTE (sales reviews the pricing). Those keywords only count on a thread where the agent itself raised the price difference, so a PO that simply says "please quote" can't trigger them.

Order (checkout.ts, orders.ts). The agent builds a cart and checkout server-side, with no browser session. It then sets the order to status 11 "Pay on Account", writes the B2B PO number and creates the invoice. That last part is needed because B2B Edition doesn't auto-invoice orders created this way.

Escalate. The session goes into the review queue, with optional Slack and email alerts. If the buyer's prices are **below** contract, the PO is also forwarded to the company's sales rep and admin, to catch stale contracts.

Staff console

- **Dashboard.** Counts of received, orders, quotes and escalations; a "touchless" percentage; the escalation call-out.
- **Sessions list.** State tabs with counts, plus columns for the reason, line count and message count.
- **Session detail.** The full email conversation, and a numbered **decision trail**: each step, the data it read and the outcome (decision-steps.ts).

- **Staff actions.** Force order or quote; reprocess (re-runs the decision against current store data but never submits); dismiss; reply on the thread (text typed by staff, no model involved); internal notes.
- **Simulator.** A dry-run or test-store run of a pasted PO, and the rules-driven conversation tab. Simulated sessions are badged as such and left out of the counters.
- **Settings.** Stored in Redis, with secrets encrypted (AES-256-GCM) and layered over environment variables. Configuration happens in the app, not on the command line.
- **Silk vs customer views.** The customer view hides token usage and AI model settings. This is enforced server-side, but it is **not access control**: anyone signed in can switch views, which is how you wanted it during testing.
- **Sign-in.** Three routes:
  - a staff secret;
  - a signed session cookie from the BigCommerce control panel;